

Solutions - Exercices Machine Learning

Régression Linéaire, KNN, K-means, Arbre de Décision

AmineGR03

5 février 2026

Table des matières

1	Régression Linéaire	2
1.1	Solution Exercice 1 : Calcul de la MSE et des paramètres	2
1.2	Solution Exercice 2 : Gradient Descent	3
1.3	Solution Exercice 3 : Questions de cours	5
2	K-Nearest Neighbors (KNN)	7
2.1	Solution Exercice 1 : Calcul manuel	7
2.2	Solution Exercice 2 : Matrice de confusion	8
2.3	Solution Exercice 3 : Questions de cours	10
3	K-Means Clustering	12
3.1	Solution Exercice 1 : Calcul manuel	12
3.2	Solution Exercice 2 : Code Python	13
3.3	Solution Exercice 3 : Questions de cours	15
4	Arbre de Décision	17
4.1	Solution Exercice 1 : Entropie et gain d'information	17
4.2	Solution Exercice 2 : Gini Impurity	19
4.3	Solution Exercice 3 : Questions de cours	20
5	Conclusion	22

Régression Linéaire

Solution Exercice 1 : Calcul de la MSE et des paramètres

Solution

Données :

X	Y	XY	X ²	Y ²
1	45000	45000	1	2025000000
2	50000	100000	4	2500000000
3	60000	180000	9	3600000000
4	80000	320000	16	6400000000
5	110000	550000	25	12100000000
Σ	345000	1195000	55	26625000000

1. Calcul de a et b :

$$n = 5, \sum X = 15, \sum Y = 345000, \sum XY = 1195000, \sum X^2 = 55$$

$$\begin{aligned}
 a &= \frac{n \sum XY - \sum X \sum Y}{n \sum X^2 - (\sum X)^2} \\
 &= \frac{5 \times 1195000 - 15 \times 345000}{5 \times 55 - 15^2} \\
 &= \frac{5975000 - 5175000}{275 - 225} \\
 &= \frac{800000}{50} \\
 &= 16000
 \end{aligned}$$

$$\begin{aligned}
 b &= \frac{\sum Y - a \sum X}{n} \\
 &= \frac{345000 - 16000 \times 15}{5} \\
 &= \frac{345000 - 240000}{5} \\
 &= \frac{105000}{5} \\
 &= 21000
 \end{aligned}$$

Modèle : $\hat{Y} = 16000X + 21000$

2. Calcul des prédictions :

X	Y réel	$\hat{Y}$	Erreur (Y - $\hat{Y}$)
1	45000	37000	8000
2	50000	53000	-3000
3	60000	69000	-9000
4	80000	85000	-5000
5	110000	101000	9000

3. Calcul de la MSE :

$$\begin{aligned}
MSE &= \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2 \\
&= \frac{1}{5} [(8000)^2 + (-3000)^2 + (-9000)^2 + (-5000)^2 + (9000)^2] \\
&= \frac{1}{5} [64000000 + 9000000 + 81000000 + 25000000 + 81000000] \\
&= \frac{260000000}{5} \\
&= 52000000
\end{aligned}$$

4. Interprétation :

- $a = 16000$: Chaque année d'expérience supplémentaire augmente le salaire de 16000€ en moyenne.
- $b = 21000$: Le salaire de base (sans expérience) est de 21000€.

Tip**Tips pour le calcul :**

- Vérifiez toujours que $\sum X^2 - (\sum X)^2/n \neq 0$ (sinon division par zéro)
- Organisez vos calculs dans un tableau pour éviter les erreurs
- La MSE est toujours positive (somme de carrés)
- Plus la MSE est petite, meilleur est le modèle

Solution Exercice 2 : Gradient Descent**Solution**

Données : $a_0 = 0, b_0 = 0, \alpha = 0.01, n = 3$

Points : (2, 4), (3, 6), (4, 8)

1. Calcul des prédictions initiales :

Pour chaque point : $\hat{Y}_i = a_0 \times X_i + b_0 = 0 \times X_i + 0 = 0$

X	Y	$\hat{Y}$	$\hat{Y} - Y$
2	4	0	-4
3	6	0	-6
4	8	0	-8

2. Calcul des dérivées partielles :

$$\begin{aligned}
\frac{\partial MSE}{\partial a} &= \frac{2}{n} \sum_{i=1}^n X_i (\hat{Y}_i - Y_i) \\
&= \frac{2}{3} [2 \times (-4) + 3 \times (-6) + 4 \times (-8)] \\
&= \frac{2}{3} [-8 - 18 - 32] \\
&= \frac{2}{3} \times (-58) \\
&= -\frac{116}{3} \approx -38.67
\end{aligned}$$

$$\begin{aligned}
\frac{\partial MSE}{\partial b} &= \frac{2}{n} \sum_{i=1}^n (\hat{Y}_i - Y_i) \\
&= \frac{2}{3} [(-4) + (-6) + (-8)] \\
&= \frac{2}{3} \times (-18) \\
&= -12
\end{aligned}$$

3. Mise à jour des paramètres :

$$\begin{aligned}
a_{new} &= a_{old} - \alpha \frac{\partial MSE}{\partial a} \\
&= 0 - 0.01 \times (-38.67) \\
&= 0.3867
\end{aligned}$$

$$\begin{aligned}
b_{new} &= b_{old} - \alpha \frac{\partial MSE}{\partial b} \\
&= 0 - 0.01 \times (-12) \\
&= 0.12
\end{aligned}$$

4. Nouvelle MSE :

Nouvelles prédictions avec $a = 0.3867$ et $b = 0.12$:

X	Y	$\hat{Y}$	$(Y - \hat{Y})^2$
2	4	0.8934	9.65
3	6	1.2801	22.30
4	8	1.6668	40.11

$$\begin{aligned}
MSE_{new} &= \frac{1}{3} [9.65 + 22.30 + 40.11] \\
&= \frac{72.06}{3} \\
&= 24.02
\end{aligned}$$

5. Explication de la convergence :

Le Gradient Descent converge car :

- Les dérivées indiquent la direction de la plus forte augmentation de l'erreur
- En allant dans le sens opposé (moins la dérivée), on réduit l'erreur
- Avec un taux d'apprentissage approprié, on converge vers le minimum
- Après plusieurs itérations, a et b se rapprochent des valeurs optimales ($a = 2$, $b = 0$ dans ce cas)

Tip

Tips pour le Gradient Descent :

- Un α trop grand peut faire diverger l'algorithme
- Un α trop petit ralentit la convergence
- Le Gradient Descent converge vers un minimum local (pas forcément global)
- Normalisez vos données pour accélérer la convergence
- Utilisez un critère d'arrêt (ex : changement de MSE $\downarrow$ seuil)

Solution Exercice 3 : Questions de cours

Solution

1. Différence régression simple vs multiple :

- **Simple** : Une seule variable indépendante : $Y = aX + b$
- **Multiple** : Plusieurs variables indépendantes : $Y = a_1X_1 + a_2X_2 + \dots + a_nX_n + b$

2. MSE comme fonction de coût :

- Pénalise fortement les grandes erreurs (carré)
- Toujours positive, facile à minimiser
- Différentiable partout
- Sensible aux outliers (valeurs aberrantes)

3. Gradient Descent :

- Algorithme d'optimisation itératif
- Calcule le gradient (dérivées) de la fonction de coût
- Met à jour les paramètres dans la direction opposée au gradient
- Répète jusqu'à convergence

4. Types de Gradient Descent :

- **Batch** : Utilise tout le dataset à chaque itération (lent mais précis)
- **Stochastique** : Un échantillon à la fois (rapide mais bruyant)
- **Mini-batch** : Compromis entre les deux (recommandé)

5. Coefficient R^2 :

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = 1 - \frac{\sum (Y_i - \hat{Y}_i)^2}{\sum (Y_i - \bar{Y})^2}$$

- $R^2 = 1$: Modèle parfait
- $R^2 = 0$: Modèle aussi bon que la moyenne
- $R^2 < 0$: Modèle pire que la moyenne
- Mesure la proportion de variance expliquée

6. Overfitting :

- Le modèle apprend par cœur les données d'entraînement
- Faible erreur d'entraînement mais forte erreur de test
- Solutions : régularisation (Ridge, Lasso), validation croisée, plus de données

K-Nearest Neighbors (KNN)

Solution Exercice 1 : Calcul manuel

Solution

Point de test : $T = (3, 2)$

1. Calcul des distances euclidiennes :

$$d(T, A) = \sqrt{(3-1)^2 + (2-2)^2} = \sqrt{4+0} = 2.00$$

$$d(T, B) = \sqrt{(3-2)^2 + (2-3)^2} = \sqrt{1+1} = 1.41$$

$$d(T, C) = \sqrt{(3-3)^2 + (2-1)^2} = \sqrt{0+1} = 1.00$$

$$d(T, D) = \sqrt{(3-4)^2 + (2-2)^2} = \sqrt{1+0} = 1.00$$

$$d(T, E) = \sqrt{(3-5)^2 + (2-3)^2} = \sqrt{4+1} = 2.24$$

2. Pour $k = 1$:

Le plus proche voisin est C ou D (distance = 1.00). Si on choisit C : classe = **Bleu**

3. Pour $k = 3$:

Les 3 plus proches voisins sont (par ordre croissant de distance) :

1. C (distance = 1.00, classe = Bleu)
2. D (distance = 1.00, classe = Bleu)
3. B (distance = 1.41, classe = Rouge)

Vote majoritaire : 2 Bleu, 1 Rouge $\rightarrow$ Classe prédite = **Bleu**

4. Pour $k = 5$:

Tous les points sont considérés :

- Rouge : A, B (2 votes)
- Bleu : C, D, E (3 votes)

Vote majoritaire : Classe prédite = **Bleu**

5. Influence du choix de k :

- k **petit (1-3)** : Modèle plus sensible au bruit, frontières complexes
- k **grand** : Modèle plus lisse, frontières simples, peut ignorer les détails locaux
- k **optimal** : Généralement $\sqrt{n}$ où n est le nombre d'échantillons
- k **impair** : Évite les égalités dans le vote majoritaire

Tip

Tips pour KNN :

- Normalisez toujours vos données (KNN est sensible aux échelles)
- Utilisez k impair pour éviter les égalités
- Testez plusieurs valeurs de k avec validation croisée
- KNN est coûteux en calcul pour de grands datasets
- Considérez des structures de données comme KD-tree pour accélérer

Solution Exercice 2 : Matrice de confusion**Solution****1. Matrice de confusion :**

	Prédit Positif	Prédit Négatif
Réel Positif	TP = 3	FN = 2
Réel Négatif	FP = 1	TN = 4

Explication des valeurs :

- **TP = 3** : Points 1, 2, 8 (correctement prédits comme Positifs)
- **TN = 4** : Points 5, 6, 7, 10 (correctement prédits comme Négatifs)
- **FP = 1** : Point 4 (prédit Positif mais réellement Négatif)
- **FN = 2** : Points 3, 9 (prédits Négatifs mais réellement Positifs)

2. Signification des métriques :

- **TP (True Positive)** : Vrais positifs - Correctement identifiés comme positifs
- **TN (True Negative)** : Vrais négatifs - Correctement identifiés comme négatifs
- **FP (False Positive)** : Faux positifs - Erreur de type I (alarme fausse)
- **FN (False Negative)** : Faux négatifs - Erreur de type II (manqué)

3. Calcul des métriques :

$$\begin{aligned}
 \text{Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN} \\
 &= \frac{3 + 4}{3 + 4 + 1 + 2} \\
 &= \frac{7}{10} = 0.70 = 70\%
 \end{aligned}$$

$$\begin{aligned}
 \text{Precision} &= \frac{TP}{TP + FP} \\
 &= \frac{3}{3 + 1} \\
 &= \frac{3}{4} = 0.75 = 75\%
 \end{aligned}$$

$$\begin{aligned}
 \text{Recall (Sensitivity)} &= \frac{TP}{TP + FN} \\
 &= \frac{3}{3 + 2} \\
 &= \frac{3}{5} = 0.60 = 60\%
 \end{aligned}$$

$$\begin{aligned}
 \text{Specificity} &= \frac{TN}{TN + FP} \\
 &= \frac{4}{4 + 1} \\
 &= \frac{4}{5} = 0.80 = 80\%
 \end{aligned}$$

$$\begin{aligned} \text{F1-Score} &= \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \\ &= \frac{2 \times 0.75 \times 0.60}{0.75 + 0.60} \\ &= \frac{0.90}{1.35} \\ &= 0.667 = 66.7\% \end{aligned}$$

4. Interprétation dans un contexte médical :

- **Precision (75%)** : Sur 4 diagnostics positifs, 3 sont corrects (1 faux positif)
- **Recall (60%)** : Sur 5 malades réels, on en détecte 3 (2 manqués - grave!)
- **Specificity (80%)** : Sur 5 personnes saines, on en identifie correctement 4
- **Accuracy (70%)** : 70% des diagnostics sont corrects

5. Quand privilégier Precision vs Recall :

- **Privilégier Precision** : Quand les faux positifs sont coûteux
 - Exemple : Spam detection (mieux vaut laisser passer un spam que bloquer un email important)
 - Exemple : Recommandation de produits (mieux vaut ne pas recommander que recommander un mauvais produit)
- **Privilégier Recall** : Quand les faux négatifs sont dangereux
 - Exemple : Détection de cancer (mieux vaut un faux positif qu'un faux négatif)
 - Exemple : Détection de fraude bancaire

Important

Important - Matrice de confusion :

- $TP + FN$ = Total des vrais positifs dans les données
- $TN + FP$ = Total des vrais négatifs dans les données
- $TP + FP$ = Total des prédictions positives
- $TN + FN$ = Total des prédictions négatives
- $TP + TN + FP + FN$ = Nombre total d'échantillons

Tip

Tips pour les métriques :

- Accuracy peut être trompeuse avec des classes déséquilibrées
- F1-Score est la moyenne harmonique de Precision et Recall
- Utilisez la courbe ROC pour visualiser le compromis Precision/Recall
- En cas de classes déséquilibrées, utilisez F1-Score plutôt qu'Accuracy

Solution Exercice 3 : Questions de cours

Solution

1. Principe de KNN :

- Algorithme de classification/regression non-paramétrique
- Pour classifier un point, trouve les k plus proches voisins
- Vote majoritaire (classification) ou moyenne (régression)
- Pas d'étape d'entraînement (lazy learning)

2. Avantages et inconvénients :

- **Avantages :**
 - Simple à comprendre et implémenter
 - Pas d'hypothèse sur la distribution des données
 - Efficace pour datasets non-linéaires
 - Pas d'entraînement nécessaire
- **Inconvénients :**
 - Coûteux en calcul (doit calculer toutes les distances)
 - Sensible aux features non pertinentes
 - Sensible aux outliers
 - Nécessite beaucoup de mémoire (stocke tout le dataset)

3. Choix de k :

- Utiliser validation croisée
- Règle empirique : $k = \sqrt{n}$ où n est le nombre d'échantillons
- k impair pour éviter les égalités
- k trop petit : surapprentissage
- k trop grand : sous-apprentissage

4. Normalisation :

- KNN utilise des distances, donc sensible aux échelles
- Normaliser (StandardScaler) pour que toutes les features aient la même importance
- Sinon, les features avec grandes valeurs dominent

5. Distances :

- **Euclidienne** : $d = \sqrt{\sum (x_i - y_i)^2}$ (sphérique)
- **Manhattan** : $d = \sum |x_i - y_i|$ (en damier)
- **Minkowski** : $d = (\sum |x_i - y_i|^p)^{1/p}$ (généralisation)
- Manhattan plus robuste aux outliers

6. Lazy learner :

- Pas d'étape d'entraînement (pas de modèle appris)
- Tout le calcul se fait au moment de la prédiction
- Opposé aux "eager learners" (SVM, arbres de décision)

7. Malédiction de la dimensionnalité :

- En haute dimension, tous les points sont équidistants
- La notion de "voisin proche" perd son sens
- Solution : réduction de dimensionnalité (PCA, feature selection)

K-Means Clustering

Solution Exercise 1 : Calcul manuel

Solution

Points : P1(1,2), P2(2,3), P3(3,1), P4(8,7), P5(9,8), P6(7,9)

Centroïdes initiaux : C1(2,2), C2(8,8)

Itération 1 - Assignment :

Calcul des distances de chaque point aux centroïdes :

$$d(P1, C1) = \sqrt{(1-2)^2 + (2-2)^2} = 1.00$$

$$d(P1, C2) = \sqrt{(1-8)^2 + (2-8)^2} = \sqrt{49 + 36} = 9.22$$

$\Rightarrow P1 \in \text{Cluster 1}$

$$d(P2, C1) = \sqrt{(2-2)^2 + (3-2)^2} = 1.00$$

$$d(P2, C2) = \sqrt{(2-8)^2 + (3-8)^2} = \sqrt{36 + 25} = 7.81$$

$\Rightarrow P2 \in \text{Cluster 1}$

$$d(P3, C1) = \sqrt{(3-2)^2 + (1-2)^2} = \sqrt{1 + 1} = 1.41$$

$$d(P3, C2) = \sqrt{(3-8)^2 + (1-8)^2} = \sqrt{25 + 49} = 8.60$$

$\Rightarrow P3 \in \text{Cluster 1}$

$$d(P4, C1) = \sqrt{(8-2)^2 + (7-2)^2} = \sqrt{36 + 25} = 7.81$$

$$d(P4, C2) = \sqrt{(8-8)^2 + (7-8)^2} = 1.00$$

$\Rightarrow P4 \in \text{Cluster 2}$

$$d(P5, C1) = \sqrt{(9-2)^2 + (8-2)^2} = \sqrt{49 + 36} = 9.22$$

$$d(P5, C2) = \sqrt{(9-8)^2 + (8-8)^2} = 1.00$$

$\Rightarrow P5 \in \text{Cluster 2}$

$$d(P6, C1) = \sqrt{(7-2)^2 + (9-2)^2} = \sqrt{25 + 49} = 8.60$$

$$d(P6, C2) = \sqrt{(7-8)^2 + (9-8)^2} = \sqrt{1 + 1} = 1.41$$

$\Rightarrow P6 \in \text{Cluster 2}$

Itération 1 - Update :

$$C1_{new} = \left(\frac{1+2+3}{3}, \frac{2+3+1}{3} \right) = (2, 2)$$

$$C2_{new} = \left(\frac{8+9+7}{3}, \frac{7+8+9}{3} \right) = (8, 8)$$

Les centroïdes n'ont pas changé! **Convergence atteinte.**

Clusters finaux :

- **Cluster 1** : P1, P2, P3 (centroïde : (2, 2))
- **Cluster 2** : P4, P5, P6 (centroïde : (8, 8))

Calcul de l'inertie :

$$\begin{aligned}
 Inertie &= \sum_{i=1}^2 \sum_{x \in C_i} \|x - \mu_i\|^2 \\
 &= [d(P1, C1)^2 + d(P2, C1)^2 + d(P3, C1)^2] + [d(P4, C2)^2 + d(P5, C2)^2 + d(P6, C2)^2] \\
 &= [1^2 + 1^2 + 1.41^2] + [1^2 + 1^2 + 1.41^2] \\
 &= [1 + 1 + 2] + [1 + 1 + 2] \\
 &= 4 + 4 = 8
 \end{aligned}$$

Si $k = 3$:

- On aurait 3 clusters au lieu de 2
- L'inertie serait probablement plus petite (meilleur ajustement)
- Mais risque de surapprentissage (overfitting)
- Utiliser la méthode du coude pour choisir k optimal

Tip

Tips pour K-means :

- L'initialisation aléatoire peut donner des résultats différents
- Utilisez K-means++ pour une meilleure initialisation
- Normalisez vos données avant clustering
- K-means suppose des clusters sphériques et de taille similaire
- L'inertie diminue toujours quand k augmente (attention au surapprentissage)

Solution Exercice 2 : Code Python

Solution

1. Fonction distance euclidienne :

```

1 import numpy as np
2
3 def euclidean_distance(point1, point2):
4     """Calcule la distance euclidienne entre deux points"""
5     return np.sqrt(np.sum((point1 - point2) ** 2))

```

2. Fonction d'assignment :

```

1 def assign_clusters(data, centroids):
2     """Assigne chaque point au cluster le plus proche"""
3     clusters = []
4     for point in data:
5         distances = [euclidean_distance(point, centroid)
6                     for centroid in centroids]
7         cluster = np.argmin(distances)
8         clusters.append(cluster)

```

```
9 return np.array(clusters)
```

3. Fonction de mise à jour des centroïdes :

```
1 def update_centroids(data, clusters, k):
2     """Update centroids as mean of points"""
3     centroids = []
4     for i in range(k):
5         cluster_points = data[clusters == i]
6         if len(cluster_points) > 0:
7             centroid = np.mean(cluster_points, axis=0)
8         else:
9             # Avoid empty clusters
10            centroid = data[np.random.randint(len(data))]
11            centroids.append(centroid)
12    return np.array(centroids)
```

4. Algorithme K-means complet :

```
1 def kmeans(data, k, max_iter=100, tol=1e-4):
2     """Complete K-means implementation"""
3     # Random initialization
4     n_samples, n_features = data.shape
5     centroids = data[np.random.choice(n_samples, k, replace=False)]
6
7     for iteration in range(max_iter):
8         # Assignment
9         clusters = assign_clusters(data, centroids)
10
11        # Update
12        new_centroids = update_centroids(data, clusters, k)
13
14        # Check convergence
15        if np.allclose(centroids, new_centroids, atol=tol):
16            print(f"Convergence after {iteration+1} iterations")
17            break
18
19        centroids = new_centroids
20
21    return centroids, clusters
22
23 # Usage example
24 data = np.array([[1, 2], [2, 3], [3, 1], [8, 7], [9, 8], [7, 9]])
25 centroids, clusters = kmeans(data, k=2)
26 print("Final centroids:", centroids)
27 print("Clusters:", clusters)
```

5. Comparaison avec sklearn :

```
1 from sklearn.cluster import KMeans
2
3 # With sklearn
4 kmeans_sklearn = KMeans(n_clusters=2, random_state=42, n_init=10)
5 clusters_sklearn = kmeans_sklearn.fit_predict(data)
6 centroids_sklearn = kmeans_sklearn.cluster_centers_
7
8 print("Sklearn centroids:", centroids_sklearn)
9 print("Sklearn clusters:", clusters_sklearn)
```

```
10 print("Inertia:", kmeans_sklearn.inertia_)
```

Tip

Tips pour le code :

- Utilisez numpy pour des calculs vectorisés (plus rapide)
- Gérer les clusters vides (réinitialiser aléatoirement)
- Utiliser un critère de tolérance pour la convergence
- Fixer random_state pour la reproductibilité
- n_init dans sklearn teste plusieurs initialisations

Solution Exercice 3 : Questions de cours

Solution

1. Principe de K-means :

1. Initialiser k centroïdes aléatoirement
2. **Assignment** : Assigner chaque point au cluster le plus proche
3. **Update** : Recalculer les centroïdes comme moyenne des points
4. Répéter 2-3 jusqu'à convergence

2. Avantages et inconvénients :

- **Avantages** :
 - Simple et rapide
 - Efficace pour grands datasets
 - Garantit la convergence
- **Inconvénients** :
 - Nécessite de connaître k à l'avance
 - Sensible à l'initialisation
 - Suppose des clusters sphériques
 - Sensible aux outliers

3. Choix de k - Méthode du coude :

- Tracer l'inertie en fonction de k
- Le "coude" (point d'inflexion) indique le k optimal
- Autres méthodes : Silhouette score, Gap statistic

4. Initialisation :

- Initialisation aléatoire peut donner des résultats différents
- K-means++ : choisit les centroïdes initiaux intelligemment
- Meilleure initialisation = convergence plus rapide et meilleurs résultats

5. Inertie :

- Somme des distances au carré entre points et centroïdes
- Mesure de compacité des clusters
- Plus petite = meilleur clustering

- Diminue toujours quand k augmente

6. Clusters vides :

- Peuvent apparaître si un centroïde est trop éloigné
- Solutions : réinitialiser, utiliser K-means++, augmenter k

7. Clustering hiérarchique vs K-means :

- **K-means** : k fixe, partitionnement plat
- **Hiérarchique** : Structure arborescente, pas besoin de k
- Hiérarchique plus lent mais plus flexible

8. Hypothèses de K-means :

- Clusters sphériques (distance euclidienne)
- Clusters de taille similaire
- Variance similaire dans chaque cluster
- Si ces hypothèses ne sont pas respectées, K-means peut échouer

Arbre de Décision

Solution Exercice 1 : Entropie et gain d'information

Solution

1. Entropie initiale :

Total : 14 jours

- Oui : 9 jours
- Non : 5 jours

$$\begin{aligned}
 \text{Entropie}(S) &= -p_{\text{Oui}} \log_2(p_{\text{Oui}}) - p_{\text{Non}} \log_2(p_{\text{Non}}) \\
 &= -\frac{9}{14} \log_2\left(\frac{9}{14}\right) - \frac{5}{14} \log_2\left(\frac{5}{14}\right) \\
 &= -0.643 \times (-0.637) - 0.357 \times (-1.485) \\
 &= 0.410 + 0.530 \\
 &= 0.940
 \end{aligned}$$

2. Entropie pour chaque attribut :

Attribut Outlook :

- **Ensoleillé** : 5 jours (2 Oui, 3 Non)

$$\begin{aligned}
 \text{Entropie}(\text{Ensoleillé}) &= -\frac{2}{5} \log_2\left(\frac{2}{5}\right) - \frac{3}{5} \log_2\left(\frac{3}{5}\right) \\
 &= 0.971
 \end{aligned}$$

- **Nuageux** : 4 jours (4 Oui, 0 Non)

$$\begin{aligned}
 \text{Entropie}(\text{Nuageux}) &= -\frac{4}{4} \log_2\left(\frac{4}{4}\right) - \frac{0}{4} \log_2\left(\frac{0}{4}\right) \\
 &= 0 \text{ (cluster pur)}
 \end{aligned}$$

- **Pluvieux** : 5 jours (3 Oui, 2 Non)

$$\begin{aligned}
 \text{Entropie}(\text{Pluvieux}) &= -\frac{3}{5} \log_2\left(\frac{3}{5}\right) - \frac{2}{5} \log_2\left(\frac{2}{5}\right) \\
 &= 0.971
 \end{aligned}$$

$$\begin{aligned}
 \text{Entropie}(S, \text{Outlook}) &= \frac{5}{14} \times 0.971 + \frac{4}{14} \times 0 + \frac{5}{14} \times 0.971 \\
 &= 0.347 + 0 + 0.347 \\
 &= 0.694
 \end{aligned}$$

Attribut Température :

- **Chaude** : 4 jours (2 Oui, 2 Non) → Entropie = 1.000
- **Douce** : 6 jours (4 Oui, 2 Non) → Entropie = 0.918
- **Fraîche** : 4 jours (3 Oui, 1 Non) → Entropie = 0.811

$$\begin{aligned}
 \text{Entropie}(S, \text{Température}) &= \frac{4}{14} \times 1.000 + \frac{6}{14} \times 0.918 + \frac{4}{14} \times 0.811 \\
 &= 0.286 + 0.393 + 0.232 \\
 &= 0.911
 \end{aligned}$$

Attribut Humidité :

- **Élevée** : 7 jours (3 Oui, 4 Non) → Entropie = 0.985
- **Normale** : 7 jours (6 Oui, 1 Non) → Entropie = 0.592

$$\begin{aligned}
 \text{Entropie}(S, \text{Humidité}) &= \frac{7}{14} \times 0.985 + \frac{7}{14} \times 0.592 \\
 &= 0.493 + 0.296 \\
 &= 0.789
 \end{aligned}$$

3. Gain d'information :

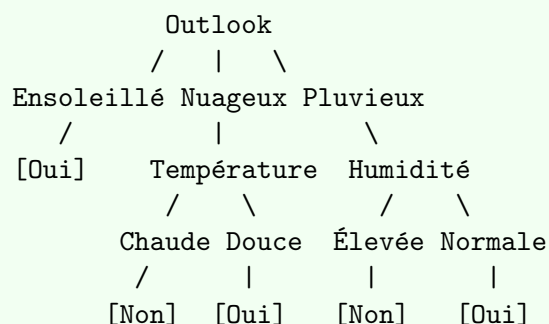
$$\begin{aligned}
 \text{Gain}(S, \text{Outlook}) &= \text{Entropie}(S) - \text{Entropie}(S, \text{Outlook}) \\
 &= 0.940 - 0.694 = 0.246
 \end{aligned}$$

$$\text{Gain}(S, \text{Température}) = 0.940 - 0.911 = 0.029$$

$$\text{Gain}(S, \text{Humidité}) = 0.940 - 0.789 = 0.151$$

4. Choix de la racine :

Outlook a le gain d'information le plus élevé (0.246) → **Racine = Outlook**

5. Construction de l'arbre :**6. Classification du nouveau jour :**

Jour : (Ensoleillé, Chaude, Normale)

1. Racine : Outlook = Ensoleillé
2. Sous-arbre Ensoleillé : Température = Chaude → **Jouer = Non**

Tip

Tips pour les arbres de décision :

- L'entropie est maximale (1.0) quand les classes sont équilibrées (50/50)
- L'entropie est minimale (0.0) quand une classe domine (cluster pur)
- Le gain d'information mesure la réduction d'incertitude
- Choisir toujours l'attribut avec le gain maximum
- Arrêter quand l'entropie = 0 (feuille pure) ou plus d'attributs

Solution Exercice 2 : Gini Impurity

Solution

1. Gini initial :

$$\begin{aligned}
 Gini(S) &= 1 - p_{Oui}^2 - p_{Non}^2 \\
 &= 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 \\
 &= 1 - 0.413 - 0.128 \\
 &= 0.459
 \end{aligned}$$

2. Gini Gain pour chaque attribut :**Attribut Outlook :**

- Ensoleillé : $Gini = 1 - \left(\frac{2}{5}\right)^2 - \left(\frac{3}{5}\right)^2 = 0.480$
- Nuageux : $Gini = 1 - \left(\frac{4}{4}\right)^2 - \left(\frac{0}{4}\right)^2 = 0$ (pur)
- Pluvieux : $Gini = 1 - \left(\frac{3}{5}\right)^2 - \left(\frac{2}{5}\right)^2 = 0.480$

$$\begin{aligned}
 Gini(S, Outlook) &= \frac{5}{14} \times 0.480 + \frac{4}{14} \times 0 + \frac{5}{14} \times 0.480 \\
 &= 0.171 + 0 + 0.171 \\
 &= 0.342
 \end{aligned}$$

$$GiniGain(S, Outlook) = 0.459 - 0.342 = 0.117$$

Attribut Humidité :

- Élevée : $Gini = 1 - \left(\frac{3}{7}\right)^2 - \left(\frac{4}{7}\right)^2 = 0.490$
- Normale : $Gini = 1 - \left(\frac{6}{7}\right)^2 - \left(\frac{1}{7}\right)^2 = 0.245$

$$\begin{aligned}
 Gini(S, Humidité) &= \frac{7}{14} \times 0.490 + \frac{7}{14} \times 0.245 \\
 &= 0.245 + 0.123 \\
 &= 0.368
 \end{aligned}$$

$$GiniGain(S, Humidité) = 0.459 - 0.368 = 0.091$$

3. Comparaison Entropie vs Gini :

- **Entropie** : $\text{Gain}(\text{Outlook}) = 0.246$, $\text{Gain}(\text{Humidité}) = 0.151$
- **Gini** : $\text{Gain}(\text{Outlook}) = 0.117$, $\text{Gain}(\text{Humidité}) = 0.091$
- Les deux critères donnent le même ordre de préférence
- Gini est plus rapide à calculer (pas de logarithme)
- Entropie est plus sensible aux différences
- En pratique, les résultats sont souvent similaires

Important

Important - Entropie vs Gini :

- Les deux mesurent l'impureté
- Entropie : $-\sum p_i \log_2(p_i)$ (plus coûteux)
- Gini : $1 - \sum p_i^2$ (plus rapide)
- Les deux donnent généralement des arbres similaires
- Gini favorise légèrement les splits équilibrés

Solution Exercice 3 : Questions de cours

Solution

1. Principe de construction :

1. Choisir l'attribut avec le gain d'information maximum
2. Créer un nœud pour cet attribut
3. Diviser le dataset selon les valeurs de l'attribut
4. Répéter récursivement pour chaque sous-ensemble
5. Arrêter quand entropie = 0 ou plus d'attributs

2. Entropie vs Gini :

- **Entropie** : Mesure l'incertitude (information)
- **Gini** : Mesure l'impureté (probabilité d'erreur)
- Gini plus rapide (pas de log)
- Résultats souvent similaires

3. Gain d'information :

- Mesure la réduction d'incertitude
- $\text{Gain} = \text{Entropie}(\text{avant}) - \text{Entropie}(\text{après})$
- Plus le gain est élevé, meilleur est le split
- Permet de choisir le meilleur attribut à chaque étape

4. Overfitting :

- L'arbre devient trop profond
- Apprend par cœur les données d'entraînement
- Mauvaise généralisation
- Solutions : pruning, limitation de profondeur, `min_samples_split`

5. Techniques de pruning :

- **Pre-pruning** : Arrêter avant (max_depth, min_samples)
- **Post-pruning** : Construire l'arbre complet puis élaguer
- Post-pruning généralement meilleur mais plus coûteux

6. Pré vs Post-élagage :

- **Pre-pruning** : Plus rapide, peut sous-apprendre
- **Post-pruning** : Plus précis, peut mieux généraliser
- Post-pruning permet de voir l'arbre complet avant décision

7. Valeurs manquantes :

- Utiliser la valeur la plus fréquente
- Utiliser la valeur moyenne (régression)
- Créer une branche "manquante"
- Utiliser des algorithmes comme C4.5 qui gèrent les valeurs manquantes

8. Avantages et inconvénients :

- **Avantages** :
 - Facile à interpréter
 - Pas besoin de normalisation
 - Gère les features non-linéaires
 - Gère les valeurs manquantes
- **Inconvénients** :
 - Sujet au surapprentissage
 - Instable (petit changement → grand changement)
 - Biais vers les features avec beaucoup de valeurs

9. Random Forest :

- Ensemble de plusieurs arbres de décision
- Chaque arbre entraîné sur un sous-échantillon (bootstrap)
- Vote majoritaire pour la prédiction finale
- Réduit le surapprentissage et améliore la robustesse
- Plus précis mais moins interprétable qu'un seul arbre

Tip

Tips pour les arbres de décision :

- Utilisez max_depth pour limiter la profondeur
- min_samples_split pour éviter les splits sur peu de données
- min_samples_leaf pour garantir un minimum dans chaque feuille
- Utilisez la validation croisée pour choisir les hyperparamètres
- Visualisez l'arbre pour comprendre les décisions
- Random Forest est souvent meilleur qu'un seul arbre

Conclusion

Ces solutions couvrent les concepts fondamentaux du Machine Learning avec des explications détaillées :

- **Régression Linéaire** : Calculs manuels, MSE, Gradient Descent avec itérations
- **KNN** : Distances, classification, matrice de confusion complète avec toutes les métriques
- **K-means** : Algorithme itératif, calculs de distances et centroïdes, code Python
- **Arbre de Décision** : Entropie, gain d'information, construction complète de l'arbre

Points clés à retenir :

1. Toujours vérifier vos calculs étape par étape
2. Comprendre l'interprétation des métriques dans le contexte
3. Pratiquer les calculs manuels pour bien comprendre les algorithmes
4. Utiliser les tips pour éviter les erreurs courantes

Bon courage pour vos révisions et examens !